Wistia Video Analytics

AWS Lambda ingestion - source code and project handoff | Version 1 | 26 October 2026

Document status

This document reconstructs the most recent Lambda code supplied in this conversation. It is not a direct export from the AWS console. Compare with the deployed lambda_function.py before using it as the definitive GitHub source.

Test evidence

• Lambda invocation: succeeded (green execution banner).

• Application response: HTTP-style statusCode 207 (partial extraction, not complete).

• FR3 and FR4: two videos successfully returned metadata and engagement metrics.

• FR5: visitor pagination retrieved records but reported complete=false and "Lambda time running low"; event extraction also incomplete.

• all_extractions_complete=false; stored_to_s3=false; errors=[].

• Observed duration: approximately 50.7 seconds. Verify Lambda timeout in Configuration > General configuration.

Implementation notes

• Credentials: token loaded at runtime from AWS Secrets Manager; no token value is included in this document.

• Authentication: Bearer was used in successful tests. The project specification also mentions Basic Auth; reconcile this discrepancy with the SME.

• FR5 visitors endpoint is account-wide; events are filtered by the two media IDs.

• Current code retains visitor and event records only during execution, returns counts/field names, and does not write to S3.

• Privacy: code avoids logging raw visitor records and IP addresses. Prior to Bronze persistence, implement IP pseudonymization with a secret salt, data minimization, and restrictive S3 access.

• Pagination is capped at 100 pages per endpoint. A complete=true value indicates a short page was received, not proof of exhaustive retrieval beyond all API edge cases.

• For GitHub: place Python source under src/lambda/lambda_function.py; add tests, README, requirements, deployment instructions and CI/CD after productionization.

Complete Lambda Python source - v1 snapshot

Filename: wistia_lambda_ingestion_v1.py | Python 3.12 | Standard library + boto3

```
import json
import boto3
import urllib.request
import urllib.error
import urllib.parse
import time
from datetime import datetime, timezone

SECRET_NAME = "wistia-video-analytics/api-token"
REGION = "us-east-1"
API_VERSION = "2026-09"
MEDIA_IDS = ["8hunphufxp", "9k4tbcdfg0"]
PAGE_SIZE = 100
MAX_PAGES = 100


def get_api_token():
    client = boto3.client("secretsmanager", region_name=REGION)
    response = client.get_secret_value(SecretId=SECRET_NAME)
    return json.loads(response["SecretString"])["WISTIA_API_TOKEN"]


def fetch_json(url, token, modern=False):
    headers = {
        "Authorization": f"Bearer {token}",
        "Accept": "application/json",
    }
    if modern:
        headers["X-Wistia-API-Version"] = API_VERSION

    for attempt in range(3):
        request = urllib.request.Request(url, headers=headers)
        try:
            with urllib.request.urlopen(request, timeout=15) as response:
                return json.loads(response.read().decode("utf-8"))
        except urllib.error.HTTPError as error:
            if error.code in (429, 500, 502, 503, 504) and attempt < 2:
                time.sleep(2 ** attempt)
                continue
            raise
        except urllib.error.URLError:
            if attempt < 2:
                time.sleep(2 ** attempt)
                continue
            raise


def fetch_all_pages(endpoint, token, context, filters=None):
    records = []
    page = 1
    complete = False
    stop_reason = None

    while page <= MAX_PAGES:
        if context.get_remaining_time_in_millis() < 10000:
            stop_reason = "Lambda time running low"
            break

        params = {"page": page, "per_page": PAGE_SIZE}
        if filters:
```

```
            params.update(filters)

        url = (
            "https://api.wistia.com/modern/stats/"
            + endpoint
            + "?"
            + urllib.parse.urlencode(params)
        )
        response = fetch_json(url, token, modern=True)

        if isinstance(response, list):
            batch = response
        elif isinstance(response, dict):
            batch = None
            for key in ("data", "results", "items"):
                if isinstance(response.get(key), list):
                    batch = response[key]
                    break
            if batch is None:
                raise ValueError("Unexpected pagination response structure")
        else:
            raise ValueError("Unexpected API response type")

        records.extend(batch)
        if len(batch) < PAGE_SIZE:
            complete = True
            break
        page += 1

    if not complete and stop_reason is None:
        stop_reason = "Maximum page limit reached"

    return {
        "records": records,
        "pages_fetched": page if complete else min(page - 1, MAX_PAGES),
        "complete": complete,
        "stop_reason": stop_reason,
    }


def safe_visitor_summary(visitor):
    return {
        "visitor_key_present": bool(visitor.get("visitor_key")),
        "ip_field_present": any(key in visitor for key in ("ip", "ip_address")),
        "event_count_available": "event_count" in visitor,
    }


def lambda_handler(event, context):
    token = get_api_token()
    metadata_results = []
    errors = []
    event_results = []

    # FR3 and FR4: media metadata and engagement
    for media_id in MEDIA_IDS:
        try:
            metadata = fetch_json(
                "https://api.wistia.com/v1/medias/" + media_id + ".json", token
            )
            stats = fetch_json(
                "https://api.wistia.com/v1/stats/medias/" + media_id + ".json",
                token,
            )
            metadata_results.append({
                "media_id": metadata.get("id"),
```

```
            "hashed_id": metadata.get("hashed_id"),
            "title": metadata.get("name"),
            "created_at": metadata.get("created"),
            "updated_at": metadata.get("updated"),
            "engagement_metrics": {
                "load_count": stats.get("load_count"),
                "play_count": stats.get("play_count"),
                "play_rate": stats.get("play_rate"),
                "hours_watched": stats.get("hours_watched"),
                "engagement": stats.get("engagement"),
            },
        })
    except Exception as error:
        errors.append({
            "stage": "media",
            "media_id": media_id,
            "error_type": type(error).__name__,
        })

# FR5: account-wide visitor pagination
visitor_result = {
    "records": [], "pages_fetched": 0,
    "complete": False, "stop_reason": "Not attempted",
}
try:
    visitor_result = fetch_all_pages("visitors", token, context)
except Exception as error:
    errors.append({"stage": "visitors", "error_type": type(error).__name__})

# FR5: per-media event pagination
for media_id in MEDIA_IDS:
    try:
        result = fetch_all_pages(
            "events", token, context, filters={"media_id": media_id}
        )
        event_results.append({
            "media_id": media_id,
            "events_extracted": len(result["records"]),
            "pages_fetched": result["pages_fetched"],
            "complete": result["complete"],
            "stop_reason": result["stop_reason"],
            "event_fields": (
                list(result["records"][0].keys())
                if result["records"] and isinstance(result["records"][0], dict)
                else []
            ),
        })
    except Exception as error:
        errors.append({
            "stage": "events",
            "media_id": media_id,
            "error_type": type(error).__name__,
        })

visitors = visitor_result["records"]
visitor_summary = {
    "visitors_extracted": len(visitors),
    "pages_fetched": visitor_result["pages_fetched"],
    "complete": visitor_result["complete"],
    "stop_reason": visitor_result["stop_reason"],
    "visitor_fields": (
        list(visitors[0].keys())
        if visitors and isinstance(visitors[0], dict)
        else []
    ),
    "first_visitor_privacy_check": (
```

```
        safe_visitor_summary(visitors[0])
        if visitors and isinstance(visitors[0], dict)
        else {}
    ),
}

all_complete = (
    len(metadata_results) == len(MEDIA_IDS)
    and visitor_result["complete"]
    and len(event_results) == len(MEDIA_IDS)
    and all(result["complete"] for result in event_results)
    and not errors
)

output = {
    "message": "Wistia FR3 FR4 FR5 extraction run",
    "extracted_at": datetime.now(timezone.utc).isoformat(),
    "media": metadata_results,
    "fr5_visitors": visitor_summary,
    "fr5_events": event_results,
    "all_extractions_complete": all_complete,
    "stored_to_s3": False,
    "errors": errors,
}
return {
    "statusCode": 200 if all_complete else 207,
    "body": json.dumps(output),
}
```